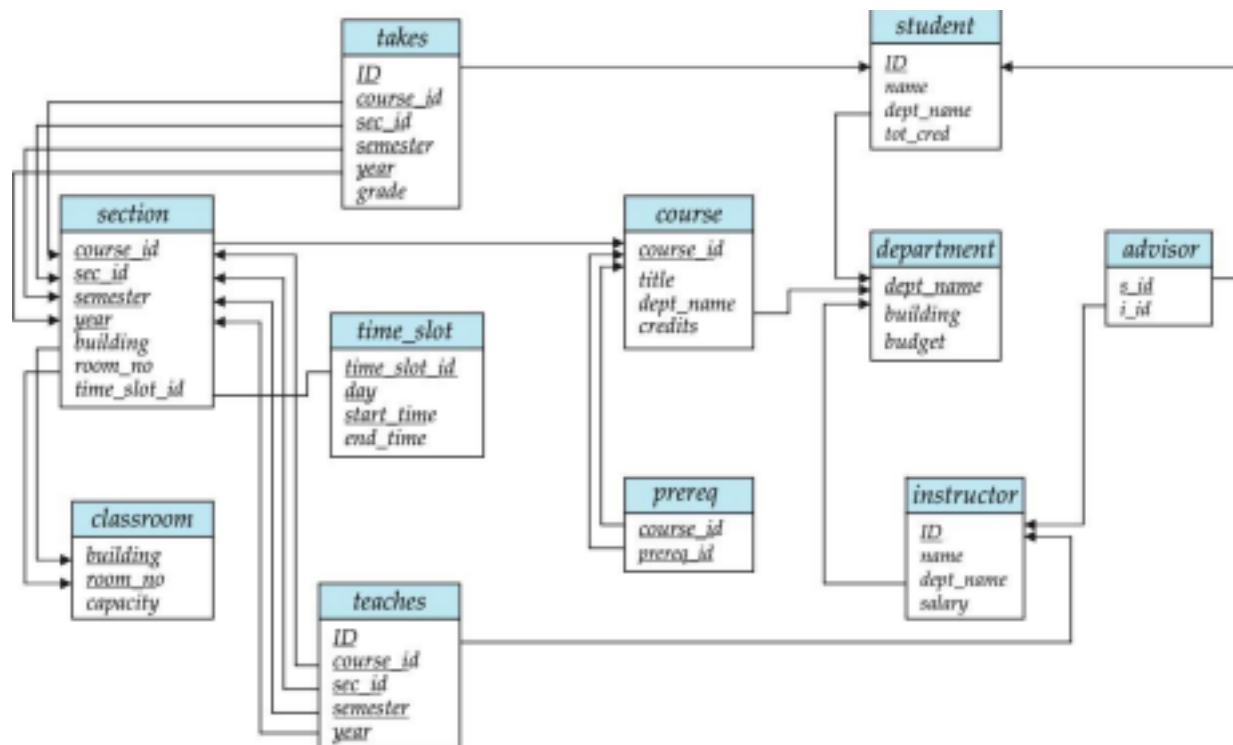


WEEK - 2 Demo

Queries on Single relations:

So far our example queries were on a single relation.

For Example: Consider a University Database



and the following query,

```
select name from instructor ;
```

where instructure table has attributes

(ID,name,dept_name, salary)

Some important cases:

- If we want to force the elimination of duplicates, we insert the keyword **distinct** after **select**. We can rewrite the preceding query as:

```
select distinct dept_name from instructor ;
```

- The **select** clause may also contain arithmetic expressions involving the operators +, −, *, and / operating on constants or attributes of tuples. For example, the query,

```
Select  
ID,name,dept_name,salary *  
1.1 from instructor ;
```

outputs a list of all the instructors, their department name and their salaries when increased by 10%.

Queries on multiple relations:

Queries often need to access information from multiple relations. We now study how to write such queries.

1. Consider the University Database (ER Diagram) given above. Suppose we want to answer the query “Retrieve the names of all instructors, along with their department names and department building name.”

Looking at the schema of the relation *instructor*, we realize that we can get the department name from the attribute *dept_name* but the department building name is present in the attribute *building* of the relation *department*.

To answer the above query, we list the relations that need to be accessed in the **from** clause, and specify the matching condition in the **where** clause.

```
select name, instructor.dept_name, building  
  
from instructor, department  
  
where instructor.dept_name = department.dept_name
```

(Note: The attribute *dept_name* required the prefix as the relation’s name (instructor or department) and others did not. Why??)

2. To find instructor names and course identifiers for instructors in the Computer Science department, we could add an extra predicate to the **where** clause, as shown below.

```
select name, course_id
from instructor, teaches
where instructor.ID= teaches.ID and
instructor.dept_name='Comp.Sci.';
```

Aliasing or Renaming:

The basic syntax of table alias is as follows –

```
select column1, column2....
from table_name as alias_name
where [condition];
```

The basic syntax of column alias is as follows –

```
select column_name as alias_name
from table_name
where [condition];
```

Example: Consider the example 1 of the above section. The same query can be written as,

```
select name, dept_name as DepartmentName, building
from instructor as ins, department as dept
where ins.dept_name = dept.dept_name
```

String Operations:

1. **String concatenation:** PostgreSQL allows you to directly concatenate strings, columns and int values using ' || ' operator. Here is the SQL query to concatenate columns first_name and last_name using ' || ' operator. You can even concatenate string with int using ' || ' operator

Example:

```
select first_name || ' ' || last_name as customer_name
from customer limit 5;
```

2. **Convert string to lowercase:** the **LOWER** function is used to convert a string, an expression, or values in a column to lowercase.

Syntax: **LOWER**(string or value or expression)

Example:

```
select LOWER(title)
from film;
```

3. **Convert string to uppercase:** Like the LOWER function, the **UPPER** function accepts a string expression or string-convertible expression and converts it to an upper case format.

Syntax: **UPPER**(string_expression)

Example:

```
select UPPER(title)
from film;
```

4. **Replace substring:** 'Replace' function searches and replaces a substring with a new substring in a string.

Syntax: **REPLACE**(source, old_text, new_text);

where;

- **source** is a string where you want to replace.
- **old_text** is the text that you want to search and replace. If the *old_text* appears multiple times in the string, all of its occurrences will be replaced.
- **new_text** is the new text that will replace the old text (*old_text*).

Example:

1. UPDATE table_name SET column_name = REPLACE(column,old_text,new_text)
WHERE condition

2. select REPLACE('ABC AA', 'A', 'Z'); // output: ZBC ZZ

5. **Length of a string** is a function that returns the number of characters in a string. It is used to determine the length of a string in terms of the number of characters it contains.

Syntax: LENGTH (string_expression)

Example:

```
select  
LENGTH('hello');  
this will return 5
```

6. **SPLIT_PART** :- This function is used to split a string into parts based on a delimiter and extract a specific part.

Syntax : SPLIT_PART(string, delimiter, part_number)

Example : SPLIT_PART('Big Adventure', ' ', 1)

This would return 'Big', because it's the first word before the space (' ').

7. **LPAD and RPAD** :- The LPAD function pads the left side of a string with a specified character until the string reaches a desired length. Similarly The RPAD function pads the right side of a string with a specified character until the string reaches a desired length.

Syntax : RPAD/LPAD(string, length, pad_char)

Example - RPAD('Big', 10, '@') it will give - Big@@@@@@@

8. **Wildcard matching** :- % operator matches any sequence of characters, including an empty sequence.

Syntax :

```
SELECT column_name FROM table_name WHERE column_name LIKE  
pattern;
```

Example :

words that **start with "J"**, such as [John](#), [Jane](#), [Jack](#).

```
SELECT * FROM employees WHERE name LIKE 'J%';
```

words that **end with "son"**, such as [Johnson](#), [Emerson](#).

```
SELECT * FROM employees WHERE name LIKE '%son';
```

words that **contain "ar"**, such as [Carl](#), [Darren](#).

```
SELECT * FROM employees WHERE name LIKE '%ar%';
```

Case-Insensitive Pattern Matching:

In standard SQL, the [LIKE](#) operator is case-sensitive. If you search for ' [son](#) ', it will not find ' [Son](#) ' or ' [SON](#) '. PostgreSQL provides [ILIKE](#) to ignore capitalization.

Syntax:

```
SELECT columns  
FROM table  
WHERE column ILIKE 'pattern';
```

Example: Find all departments where the name contains "bio" (case-insensitive).

```
SELECT *  
FROM department  
WHERE dept_name ILIKE '%bio%';
```

Note: The [%](#) symbols are wildcards representing any sequence of characters before or after the word.

Ordering

SELECT

```

        select_list
    FROM
        table_name
    ORDER BY
        sort_expression1 [ASC | DESC],
        ...
        sort_expressionN [ASC | DESC];

```

When you query data from a table, the **SELECT** statement returns rows in an unspecified order. To sort the rows of the result set, you use the **ORDER BY** clause.

- First, specify a sort expression, which can be a column or an expression, that you want to sort after the **ORDER BY** keywords. If you want to sort the result set based on multiple columns or expressions, you need to place a comma (,) between two columns or expressions to separate them.
 - Second, you use the **ASC** option to sort rows in ascending order and the **DESC** option to sort rows in descending order. If you omit the **ASC** or **DESC** option, the **ORDER BY** uses **ASC** by default.

Eg:- The below query

```

select ID, name, dept_name, salary * 1.1 as new_salary
from instructor

```

Order by new_salary **DESC**, name **ASC**;

The result of the above query, the results will be first sorted by the new_salary in the descending order, for those rows where new_salary is the same, they will be sorted by the instructor name in ascending order.

Logical Operator Precedence (Parentheses)

When mixing **AND** and **OR** in a **WHERE** clause, SQL processes **AND** first. To ensure your logic is correct, always use **parentheses** to group **OR** conditions.

Example: Find instructors who are either in the 'Physics' or 'Music' department **AND** have a salary greater than 70,000.

- **Correct Logic:**

```
SELECT name
FROM instructor
WHERE (dept_name = 'Physics' OR dept_name = 'Music')
AND salary > 70000;
```

Incorrect Logic (Returns ALL Physics instructors regardless of salary):

```
SELECT name
FROM instructor
WHERE dept_name = 'Physics' OR dept_name = 'Music'
AND salary > 70000;
```

Restoring Database

The lab from now onwards will utilize the dvdrental database.

Download the database tar file from

<https://github.com/imkumaraju/dvdrental-sample-database>.

CREATE DATABASE dvdrental;

Use the pg_restore tool to load data into the dvdrental database that we had just created as using the command:

You have to run this command outside the database

createdb -U postgres dvdrental

pg_restore -U postgres -d dvdrental [path_to_tar_file]

The next page contains the schema of dvdrental database.

